

# Graph Algorithms

## 2. Classical polynomial-time solvable problems

### 2.6 Matchings in general graphs (briefly)

---

Martin Milanič  
martin.milanic@upr.si  
UP FAMNIT

October 30, 2025



In the previous lectures, we discussed some important applications of the Max-Flow Min-Cut Theorem and maximum flow algorithms:

- A classical connectivity result: Menger's Theorem
- Computing vertex- and edge-connectivity in graphs
- Matchings and vertex covers in bipartite graphs (Kőnig-Egerváry Theorem)

In particular, we can efficiently find maximum matchings and minimum vertex covers in bipartite graphs.

A maximum matching can also be found in polynomial time  
**in general graphs.**

- J. Edmonds,  $\mathcal{O}(n^2(n + m))$ , *Paths, trees and flowers*, 1965.

**However:**

The *minimum vertex cover* problem (which is solvable in polynomial time for bipartite graphs) is **NP-hard** for general graphs.

## 2.6 Matchings in general graphs (briefly) – Contents

1. A high-level overview of Edmonds' algorithm for maximum matchings in general graphs, a quiz
2. A scheduling application of maximum matchings

Edmonds' algorithm is based on the following characterization of maximum matchings:

**Theorem (Petersen 1891, Berge 1957)**

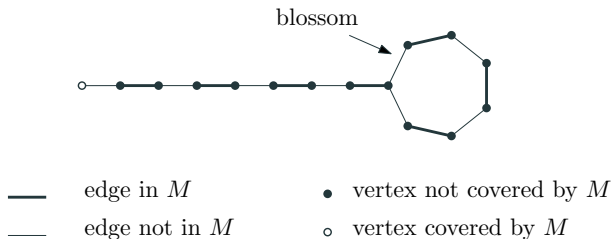
*A matching  $M$  in a graph  $G$  is maximum  $\iff G$  has no  $M$ -augmenting path.*

An  $M$ -augmenting path is a path in the graph that looks as follows:



Edmonds' polynomial-time algorithm for this problem is based on searching for augmenting paths.

Searching for such a path using BFS can be unsuccessful and can end up in a so-called “**blossom**”.



In this case the blossom is shrunk to a single vertex and the problem is solved recursively in the smaller graph.

### Theorem (Edmonds, 1965)

*There is an algorithm running in time  $\mathcal{O}(n^2(n + m))$  that correctly solves the maximum matching problem in general graphs.*

Faster algorithms are known, e.g., running in time  $\mathcal{O}(\sqrt{n}(m + n))$  due to various authors (Micali and Vazirani 1980, Blum 1990, Vazirani 1990,1994, Gabow and Tarjan 1991)

The following two problems are also solvable in polynomial time:

- The problem of finding a maximum weight matching in a given graph  $G = (V, E)$  with edge weight function  $w : E \rightarrow \mathbb{Q}_+$ .
- The problem of finding a minimum or maximum weight **perfect** matching in a given graph  $G = (V, E)$  with edge weight function  $w : E \rightarrow \mathbb{Q}_+$ .

The problems are solvable in time  $\mathcal{O}(n^2(n + m))$  (which matches the running time of Edmonds' algorithm).

Faster algorithms for these problems are known, running, e.g., in time  $\mathcal{O}(n^3)$ , and others (see, e.g., Schrijver, chapter 26).



**A quiz**

Let  $\nu(G)$  denote the maximum size of a matching in  $G$ .

A **maximum matching** in  $G$  is a matching with size  $\nu(G)$ .

For a graph  $G$  with at least one edge, let  $\nu_2(G)$  be the largest integer  $k$  such that  $k < \nu(G)$  and  $G$  has a matching with size  $k$ .

A **second largest matching** is a matching with size  $\nu_2(G)$ .

Give a polynomial-time algorithm to compute a second largest matching in a given graph  $G$ .

**Solution:**

Compute a maximum matching  $M$ , let  $e \in M$  (such an edge exists, since  $G$  has at least one edge), and let  $M' = M \setminus \{e\}$ .

Then  $M'$  is a matching such that  $|M'| = \nu(G) - 1$ .

Since  $\nu_2(G) \leq \nu(G) - 1$ , we infer that  $\nu_2(G) \leq |M'| < \nu(G)$ .

Hence,  $M'$  is a second largest matching in  $G$ .

**Matchings in general graphs**  
– **an example application**  
**(optional material)**

Suppose that we have to carry out a set of tasks, where some of the tasks must be done before others.

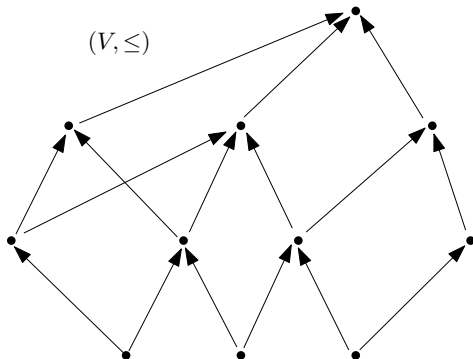
Such dependencies can be displayed with a acyclic digraph  $(V, E)$ , similarly as with the known example:



This digraph is transitive ( $(u, v) \in E$  and  $(v, w) \in E$  imply  $(u, w) \in E$ ) and therefore corresponds to a partially ordered set  $(V, \leq)$ .

Suppose that we have to carry out a set of tasks, where some of the tasks must be done before others.

Such dependencies can be displayed with a partially ordered set  $(V, \leq)$ :



Suppose that every task lasts a single time unit, say one hour.

We have two workers who can perform the tasks. Each of them can only perform one task at the time.

Equivalently: we have a machine that can carry out any two tasks at once.

We would like to carry out the given tasks **as quickly as possible**.

We reduce the problem to the **maximum matching problem in a derived graph**.

Let  $G = (V, E)$  be the graph where the vertex set  $V$  is the set of all tasks and the edge set is

$$E = \{\{u, v\} : u \not\leq v \text{ and } v \not\leq u\}.$$

*(That is, two tasks are joined by an edge if and only if they can be carried out simultaneously.)*

$G = (V, E)$  is the complement of the so-called **comparability graph** of the partially ordered set  $(V, \leq)$ .



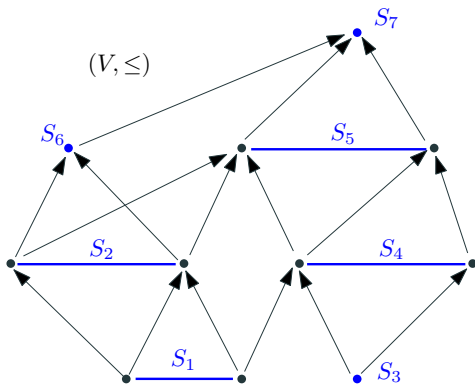
Let  $S_i$  denote the set of tasks carried out at time  $i$ .

We have a sequence  $S_1, \dots, S_t$  such that  
each  $S_i$  is either a vertex or an edge of  $G$ ,

$S_1, \dots, S_t$  form a partition of set  $V$

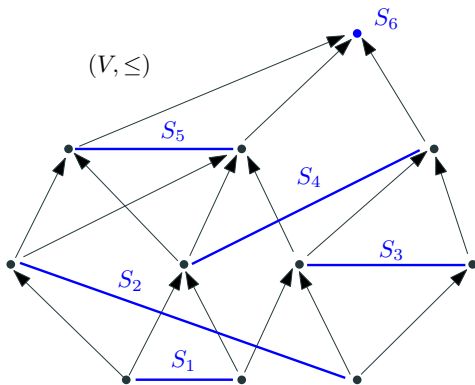
and if  $u < v$ ,  $u \in S_i$  and  $v \in S_j$ , then  $i < j$ .

Example:



Is this an optimal solution?

No! 6 hours suffice:



Sets from the sequence  $S_1, \dots, S_k$  that are pairs (that is, edges) form a matching  $M$  in  $G$ .

Consequently:  $t = |V| - |M|$ .

Thus, if we set

$\nu(G)$  = maximum size of a matching in  $G$ ,

then  $t \geq |V| - \nu(G)$ .

It turns out that there always exists a sequence of carrying out the tasks such that  $t = |V| - \nu(G)$ .

This is a consequence of the following:

### Theorem

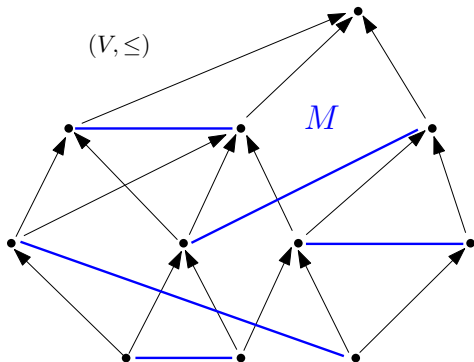
$G$  has a maximum matching  $M = \{e_1, \dots, e_t\}$  such that for all  $u \in e_i$  and  $v \in e_j$  we have:

$$u < v \Rightarrow i < j.$$

**Proof:**

By induction on  $|V|$ .

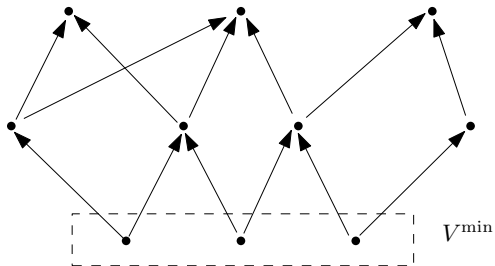
Let  $M$  be a maximum matching in  $G$ .



We may assume that  $M$  covers all vertices: otherwise, we delete the uncovered vertices and apply the induction hypothesis.

Let  $V^{\min}$  be the set of minimal elements of  $(V, \leq)$ .

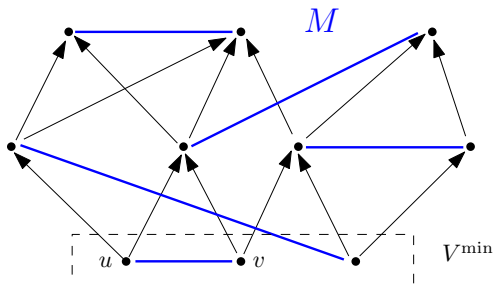
$(V, \leq)$



## Example:

If  $V^{\min}$  contains an edge  $uv \in M$  (as a subset),

$$(V, \leq)$$

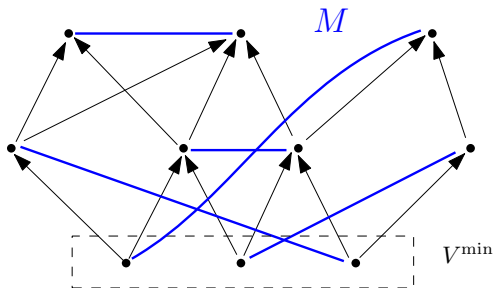


then we can delete  $u$  and  $v$  from  $V$  and apply the induction hypothesis.

## Example:

Suppose now that  $V^{\min}$  does not contain any edge  $uv \in M$  as subset.

$(V, \leq)$



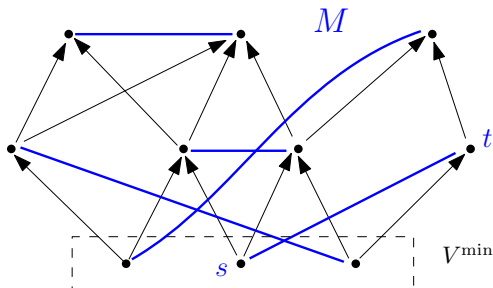
Then every vertex from  $s \in V^{\min}$  is covered with some edge  $st \in M$  such that  $t \notin V^{\min}$ .



Choose an edge  $st$  such that  $t$  is as low as possible

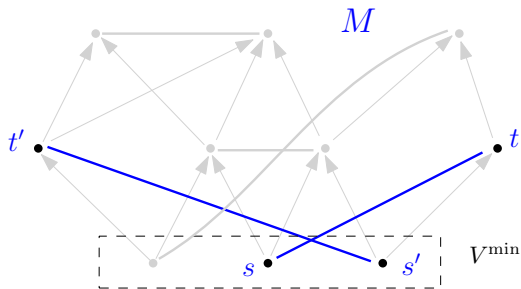
(the *height* of an element  $t =$  length of a longest directed path ending in  $t$ ).

$(V, \leq)$



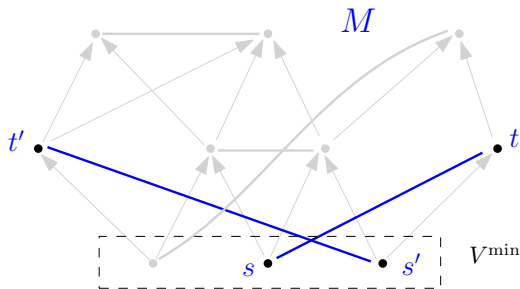
Since  $t \notin V^{\min}$ , there exists an edge  $s't' \in M$  such that  $s' \in V^{\min}$  in  $s' < t$ .

$(V, \leq)$



Since  $s$  and  $s'$  are minimal elements,  $ss'$  is an edge in  $G$ .

$$(V, \leq)$$

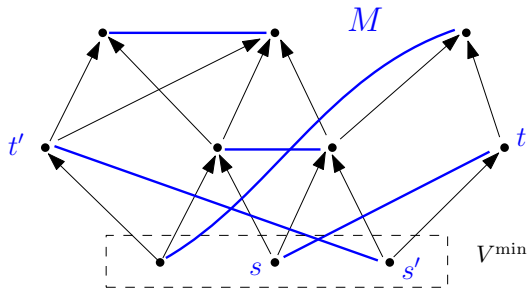


We claim that  $tt'$  is also an edge in  $G$ :

- If  $t < t'$ , then we would have  $s' < t < t'$ , contrary to the fact that  $s't' \in E$ .
- If  $t' < t$ , then  $t'$  would be lower than  $t$ , contrary to the choice of  $t$ .

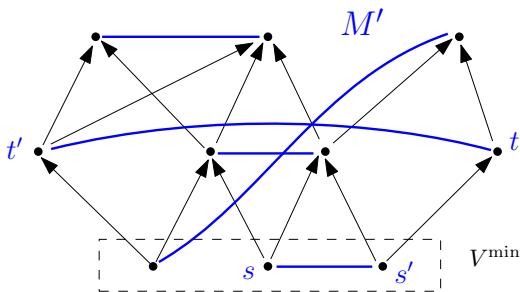
Therefore, we can replace the edges  $st$  and  $s't'$  in  $M$  with edges  $ss'$  in  $tt'$ .

$(V, \leq)$



Therefore, we can replace the edges  $st$  and  $s't'$  in  $M$  with edges  $ss'$  in  $tt'$ .

$$(V, \leq)$$



We delete  $s$  and  $s'$  from  $V$  and apply the induction hypothesis as before.  $\square$

The above argument can be turned into a polynomial-time algorithm for finding an optimal sequencing of tasks.